

Výjimky

Karel Richta a kol.

Katedra technických studií
Vysoká škola polytechnická Jihlava

© Karel Richta, Martin Hořeňovský, Aleš Hrabalík, 2021

Objektově-orientované programování, OOP 04/2021, Lekce 9

<https://moodle.vspj.cz/course/view.php?id=201479>

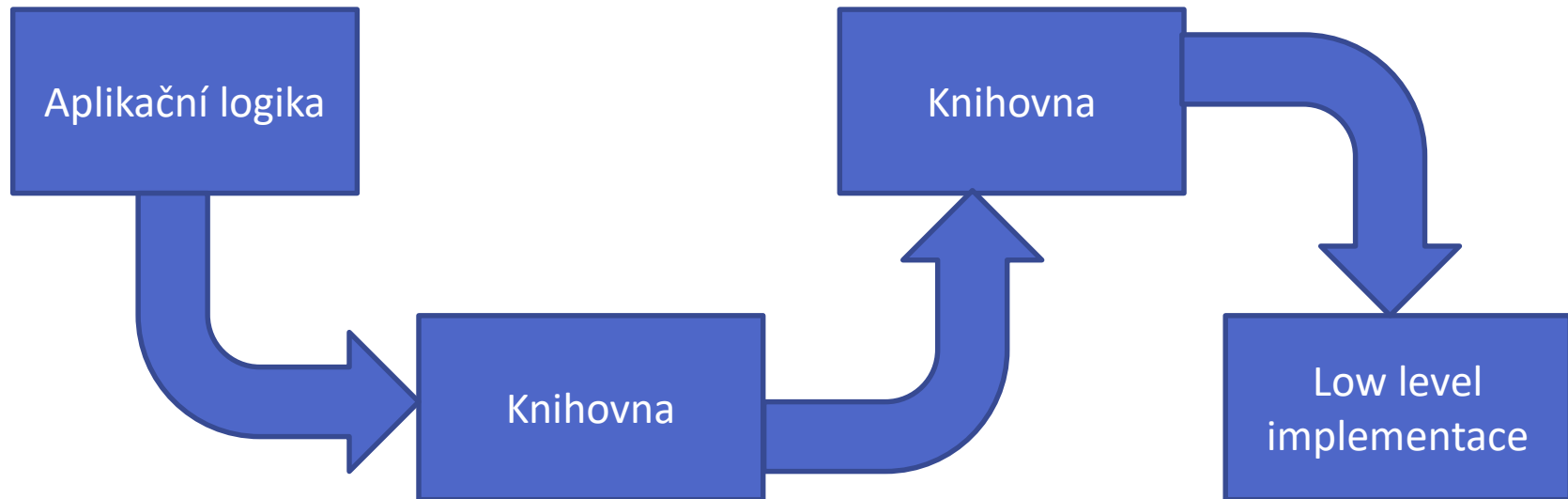


Vysoká škola
polytechnická
Jihlava



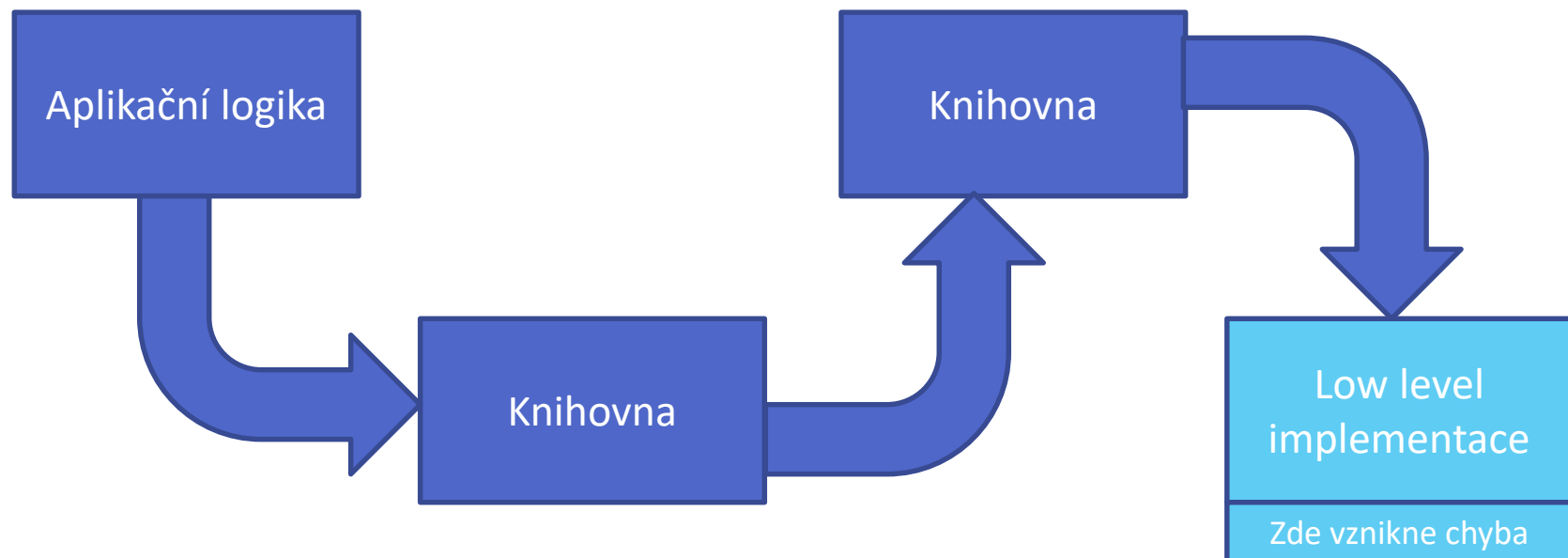
Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



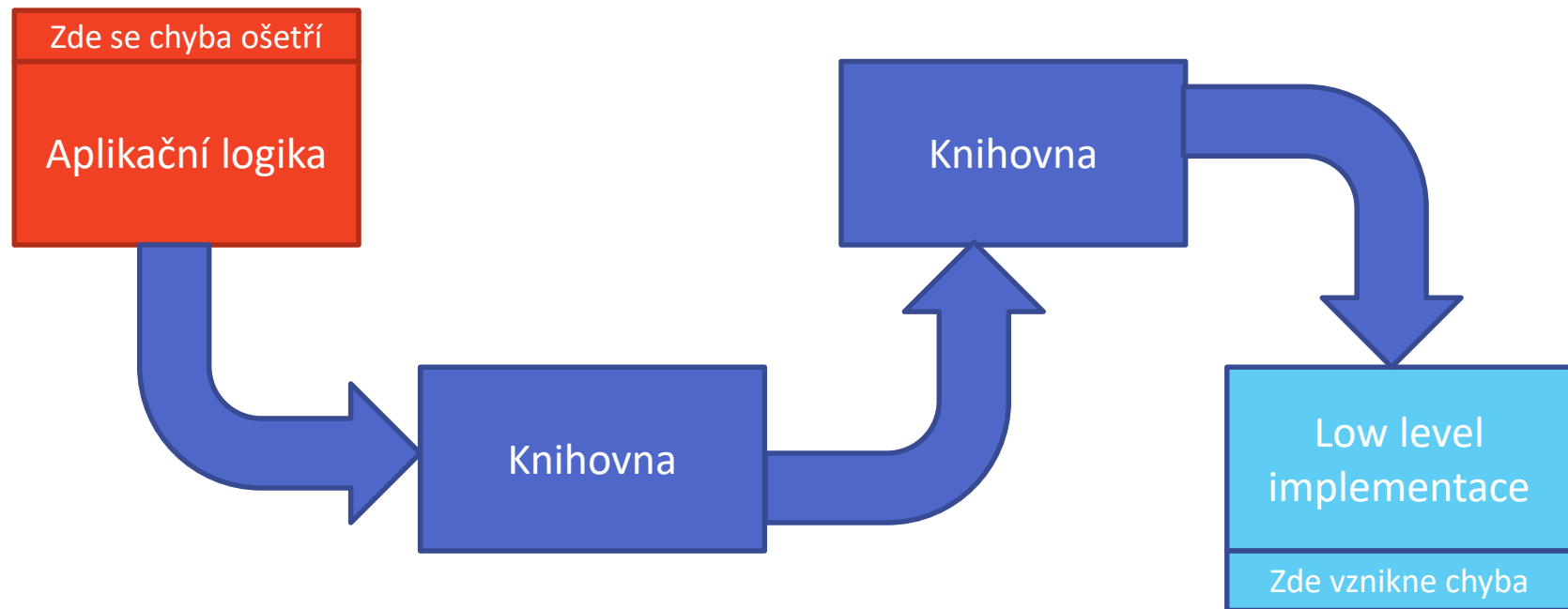
Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



Problém

- Při běhu programu mohou vznikat různé chyby.
 - Například nemusíme mít práva ke čtení/zápisu souboru.
- Tyto chyby je potřeba nějak ošetřit.
- Místo, které si umí s chybou poradit, je obvykle jiné, než místo, kde chyba vznikla.



Důsledky

- K řešení chyb je tedy potřeba přenést informace o chybě z nižších vrstev do vyšších.
- Existují 3 různé způsoby:
 - Návratové hodnoty
 - Chybové příznaky
 - Výjimky
- Ve standardní knihovně C++ se vyskytují všechny tři.
 - Části převzaté z C obvykle používají návratové hodnoty.
 - Matematické funkce používají chybové příznaky.
 - STL používá výjimky.

Řešení pomocí chybových příznaků

- Základní schema:

```
int chyba = 0; // globální chybový příznak

void f(parametry) {
    // část kódu, kde může nastat chyba
    // pokud nastane:
    // - nastaví se chybový příznak na dohodnutou nenulovou hodnotu
    // - přeruší se normální zpracování a funkce se ukončí
    // pokud chyba nenastane, nic se neděje, příznak se nenastaví
};

int main() {
    f(skutečné parametry);
    if (chyba) { zpracování chyby }
    // normální zpracování
}
```

Chybové příznaky

- Funkce, která narazí na chybu, ji zapíše do globální proměnné.
- Funkce, která se chce o chybě dozvědět, zkontroluje zapsanou hodnotu.
- Chybové příznaky se používají v matematické knihovně a v POSIX standardu.
 - Např. pokud funkce `log` narazí na chybu, třeba negativní vstup, zapíše do `errno` hodnotu `EDOM`.
 - Funkce volající `log` má následně možnost `errno` přečíst.
- Nemění signaturu funkce.

```
int main() {  
    double foo = std::log(-1.23);  
    if (errno == EDOM) {  
        std::cout << "log() has failed because: " << std::strerror(errno) <<  
        '\n';  
    }  
}
```

```
>>> log() has failed because: Numerical argument out of domain
```

Chybové příznaky – knihovna errno.h

- Definuje makro `errno` – lze si představit jako deklaraci proměnné `int errno = 0;`
- Obsahuje kód poslední zjištěné chyby (0 znamená OK).
- Nejméně tři konstanty - kódy: EDOM (domain error), ERANGE (range error) a EILSEQ (illegal sequence).
- Pokud některá funkce narazí na chybu, např. log na negativní vstup, zapíše do `errno` kód chyby, zde např. hodnotu EDOM.
- Funkce volající log má následně možnost `errno` přečíst.

```
enum class errc;
```

```
http://en.cppreference.com/w/cpp/error/errc
```


Chybové příznaky – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.
- **Není jisté, kdy a kde k chybě došlo.**
 - Pokud po volání funkce nekontrolujeme `errno`, už se nedá určit, zda k chybě došlo.
 - Ani není možné zjistit, jestli se chyb nevyskytlo více.
- **Je to spousta kódu, která se špatně čte a špatně píše.**
 - Volání funkce následuje mnoho řádků, které čtou `errno` a reagují na chybu.
 - Funkce se dají skládat, ale pouze za cenu možného přehlédnutí chyby.

```
double sumlogs(const std::vector<double>& numbers) {  
    double result = 0;  
    for (auto n : numbers) {  
        result += std::log(n);  
    }  
    return result;  
}
```

Chybové příznaky – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.

```
double sumlogs(const std::vector<double>& numbers) {  
    double result = 0;  
    for (auto n : numbers) {  
        result += std::log(n);  
    }  
    return result;  
}  
  
double foo(const std::vector<double>& numbers) {  
    return std::exp(sumlogs(numbers));  
}
```

- Když zavoláme `std::log` s neplatným parametrem, nastaví se `errno`.
To samé platí o `std::exp`.
- Došlo k chybě?
 - Nevíme.

Chybové příznaky – Problémy

- **Není jisté, kdy a kde k chybě došlo.**
 - Pokud po volání funkce nekontrolujeme `errno`, už se nedá určit, zda k chybě došlo.

```
// Toto už zkontroluje chybu, ale neví kde nastala
double sumlogs(const std::vector<double>& numbers) {
    double result = 0;
    for (auto n : numbers) {
        result += std::log(n);
    }
    if (errno == EDOM) {
        // handle error
    }
    return result;
}
```

- Chybu kontrolujeme, ale nevíme, kdy k ní došlo.
 - V tomto případě to nevadí, ale jindy může.

Chybové příznaky – Problémy

- Je to spousta kódu, která se špatně čte a špatně píše.
 - Volání funkce následuje mnoho řádků, které čtou errno a reagují na chybu.
 - Funkce se dají skládat, ale pouze za cenu možného přehlédnutí chyby.

```
double sumlogs(const std::vector<double>& numbers) {  
    double result = 0;  
    for (auto n : numbers) {  
        result += std::log(n);  
    }  
    if (errno == EDOM) {  
        std::cerr << "Doslo k chybe v sumlogs: EDOM\n";  
        return NAN;  
    }  
    return result;  
}  
  
double foo(const std::vector<double>& numbers) {  
    auto temp = sumlogs(numbers);  
    if (errno == EDOM || isnan(temp)) {  
        std::cerr << "foo chyba: suma logaritmu skončila chybou\n";  
        return NAN;  
    }  
    auto res = std::exp(temp);  
    if (errno == ERANGE) {  
        std::cerr << "foo chyba: vysledek je prilis velky.\n";  
        return NAN;  
    }  
    return res;  
}
```

Zvětšeno na příštím
snímku

Chybové příznaky – Problémy

```
double sumlogs(const std::vector<double>& numbers) {
    double result = 0;
    for (auto n : numbers) {
        result += std::log(n);
    }
    if (errno == EDOM) {
        std::cerr << "Doslo k chybe v sumlogs: EDOM\n";
        return NAN;
    }
    return result;
}

double foo(const std::vector<double>& numbers) {
    auto temp = sumlogs(numbers);
    if (errno == EDOM || isnan(temp)) {
        std::cerr << "foo chyba: suma logaritmu skončila chybou\n";
        return NAN;
    }
    auto res = std::exp(temp);
    if (errno == ERANGE) {
        std::cerr << "foo chyba: vysledek je prilis velky.\n";
        return NAN;
    }
    return res;
}
```

Chybové příznaky – Problémy

```
double sumlogs(const std::vector<double>& numbers) {  
    double result = 0;  
    for (auto n : numbers) {  
        result += std::log(n);  
    }  
    if (errno == EDOM) {  
        std::cerr << "Doslo k chybe v sumlogs: EDOM\n";  
        return NAN;  
    }  
    return result;  
}  
  
double foo(const std::vector<double>& numbers) {  
    auto temp = sumlogs(numbers);  
    if (errno == EDOM || isnan(temp)) {  
        std::cerr << "foo chyba: suma logaritmu skončila chybou\n";  
        return NAN;  
    }  
    auto res = std::exp(temp);  
    if (errno == ERANGE) {  
        std::cerr << "foo chyba: vysledek je prilis velky.\n";  
        return NAN;  
    }  
    return res;  
}
```

Řešení pomocí návratových hodnot

- Základní schema:

```
int f(parametry) {  
    // část kódu, kde může nastat chyba  
    // pokud nastane:  
    // - přeruší se normální zpracování a funkce se ukončí  
    // - návratová hodnota indikuje chybu  
    // pokud chyba nenastane, funkce vrátí hodnotu 0  
};  
  
int main() {  
    if (f(skutečné parametry)) { zpracování chyby }  
    // normální zpracování  
}
```

Řešení pomocí návratových hodnot

- Funkce, které vrací numerickou hodnotu (tradičně `int`)
 - Návratová hodnota je pak určuje chybu, ke které došlo při průběhu funkce.
 - Hodnoty značící chybu (nebo úspěch) jsou předem definovány.
- Funkce často přijímají i tzv. výstupní parametr.
- Navracené hodnoty z funkce se pak vrací výš.

```
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n;  
    int res = scanf("%d", &n);  
    if (res != 1) {  
        printf("To neni cislo\n");  
        exit(1);  
    }  
    ...  
}
```


Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Programátor musí vynaložit úsilí, aby ignorovány nebyly.
- Pokud někdy ignorujete chybu, její hlášení je navždy ztraceno.
- Je to spousta kódu, která se špatně čte a špatně píše.
 - Vrácená hodnota z každé funkce se musí uložit, zkontrolovat, vyřešit a případně vrátit z funkce.
 - Funkce se nedají skládat.

```
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n;  
    scanf("%d", &n);  
    for (int i = 0; i < n; ++i) {  
        ...  
    }  
}
```

Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Pokud někdy ignorujete chybu, její hlášení je navždy ztraceno.

```
int nacti_cislo() {  
    int temp;  
    scanf("%d", &temp);  
    return temp;  
}  
  
int main() {  
    printf("Kolik cisel bude nasledovat?\n");  
    int n = nacti_cislo();  
    ...  
}
```

- Pokud se nepovedlo načíst číslo, v n je podivná hodnota.
 - Zbytek programu o tom neví.

Návratové hodnoty – Problémy

- Chyby mohou být ignorovány.
 - Programátor vynaložit úsilí, aby ignorovány nebyly.

```
int nacti_kladne_cislo() {  
    int temp;  
    int res = scanf("%d", &temp);  
    if (res != 1) {  
        return -1;  
    }  
    if (temp < 0) {  
        return -2;  
    }  
    return temp;  
}
```

- Už nejsme schopni vrátit všechny možné hodnoty datového typu `int`.
 - Musíme vybrat, které hodnoty obětujeme.

Návratové hodnoty – Problémy

- Je to spousta kódu, který se špatně čte i píše.
 - Vrácená hodnota z každé funkce se musí uložit a zkontrolovat; v případě chyby musíme správně zareagovat.
 - Funkce se nedají skládat.

```
int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}
```

Zvětšeno na příštím
snímku

```
int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}
```

```

int nacti_kladne_cislo() {
    int temp;
    int res = scanf("%d", &temp);
    if (res != 1) {
        return -1;
    }
    if (temp < 0) {
        return -2;
    }
    return temp;
}

int main() {
    printf("Kolik cisel bude nasledovat?\n");
    int cislo = nacti_kladne_cislo();
    if (cislo < 0) {
        if (cislo == -2) {
            printf("Potrebuju KLADNE cislo.\n");
        }
        if (cislo == -1) {
            printf("Potrebuju cislo.\n");
        }
        exit(1);
    }
    ...
}

```

Řešení pomocí výjimek

- Základní schema:

```
int main() {  
    try {  
        // část kódu, kde může nastat chyba  
        // pokud nastane - způsobí změnu ve zpracování, tzv. „hodí výjimku“  
        // - přeruší se normální zpracování, hledá se ovladač této výjimky  
        // - výjimka je identifikována pomocí typu  
        // pokud chyba nenastane, nic se neděje  
    } catch (... typ výjimky...) {  
        // zpracování výjimky daného typu  
    } catch (... jiný typ výjimky...) {  
        // zpracování výjimky jiného typu  
    }  
    // další případné ovladače  
}
```

Výjimky a jejich zpracování

- Motivace výjimek
- Provádění výjimek
- Typy výjimek
- Přetypování
- RTTI

Princip výjimek

- Výjimka je reprezentována hodnotou "libovolného" typu.
- Výjimky jsou identifikovány (rozlišovány) svým typem.
- Blok try může mít několik ovladačů výjimek (catch); procházejí se sekvenčně:

```
try {  
    // zde vznikne výjimka  
}  
catch (int param) { cout << "int exception: " << param; }  
catch (char param){ cout << "char exception: " << param; }  
catch (...) { cout << "default exception"; } // catch all
```

ostatní typy výjimek

Výjimky

- Situace neumožňují pokračovat v běhu programu:
 - Nedefinovaný výsledek operace(/0)
 - Nepovolený přístup do paměti
 - Nedostatek paměti
- Může nastat v libovolném místě(metodě) a musí se řešit:
 - uvolnit nepotřebnou paměť a zkusit znovu
 - Vypsát - nelze dělit 0 a načíst nové argumenty
- Lze řešit individuálně v každém místě potenciálního nebezpečí – nárok na přehlednost a objem kódu

Výjimky – motivační příklad

```
float podil (int a, int b);

int _tmain(int argc, _TCHAR* argv[]) {
    int x,y;
    cout << "Zadej dve cela cisla: ";
    cin >> x >> y;
    cout << podil(x,y);
    // další kód
    cout << "konec programu";
};

float podil (int a, int b) {
    //if (b ==0) exit(1);
    return a/(float)b;
};
```

Po odstranění // je odstraněn problém dělení nulou,
ale stále nelze provést návrat do programu pro pokračování kódu

Výjimky - syntaxe

- Oblast která může vyvolat výjimku:
 - klíčové slovo **try**.
- Vyvolání výjimky:
 - klíčové slovo **throw**, následované dat typem, nebo objektem, definující typ výjimky.
- Oblast zachycení výjimky:
 - klíčové slovo **catch** s argumentem odpovídající dat typu, nebo objektu (předáno referencí), který byl použit pro vyvolání výjimky.
- Minimálně jeden blok, může být i více.

Výjimky - provádění

- Při výskytu problému se vyvolá výjimka:
 - Programátor vyvolává sám pomocí `throw dat_typ`.
 - Použití `new`, knihovny `stl`.
 - Objekt s informací o problému (třída `exception`).
- Výjimka najde nejbližší blok `catch`:
 - Skok na podprogram, který je celý proveden.
 - Odpovídající typu, který ji vyvolal.
- V nalezeném bloku proběhne kód reakce:
 - Po provedení následuje návrat.
 - Ne na místo, kde byla vyvolána výjimka.
 - Návrat na konec odpovídajícího bloku `try`.

Výjimky - příklad

```
float podil (int a, int b);

int _tmain(int argc, _TCHAR* argv[]) {
    int x,y;
    cout << "Zadej dve cela cisla: ";
    cin >> x >> y;
    try {
        cout<<podil(x,y);
    }
    catch(int i) {
        cout << "nelze delit nulou";
        //dokonce if (i == 1) cout<<".....";
    }
    // další kód
    cout << "konec programu";
};

float podil (int a, int b) {
    if (b ==0) throw 1;
    return a/(float)b;
};
```

Příklad řešení pomocí výjimek

```
int gcd ( int a, int b )
{
    if ( a <= 0 || b <= 0 ) throw "Invalid arguments";
    while ( a != b )
        if ( a > b ) a -= b; else b -= a;
    return ( a );
}

int a, b, c;
try {
    cin >> a >> b;
    c = gcd ( a, b );
}
catch ( const char * Err ) { cout << Err << endl; }
catch ( ... ) { /* ostatní vyjimky */ }
```

Šíření výjimek

Funkce/metoda může deklarovat, že se z ní mohou šířit výjimky:

- Může šířit libovolné výjimky:

```
int foo ( void )
```

- Může šířit pouze výjimky `Exc1` nebo `Exc2`:

```
int foo ( void ) throw (Exc1, Exc2)
```

- Nemůže šířit žádné výjimky:

```
int foo ( void ) throw ( )
```


Provádění výjimek

- Výjimka v konstruktoru:
 - provádění konstruktoru se pozastaví,
 - nová instance není inicializovaná,
 - nevolá se na ni destruktork.
- Výjimka v destruktorku:
 - ukončí se kód destruktorku,
 - provedou se destruktory ostatních lokálních objektů,
 - teprve pak se hledá ovladač výjimky.
- Výjimka v průběhu šíření jiné výjimky:
 - okamžité ukončení programu.

Standardní výjimky

- Standardní knihovna poskytuje základní třídu pro výjimky (jmenuje se exception, je definována v <exception>) vhodnou pro vytváření odvozených uživatelských výjimek.
- Tato třída mimo jiné definuje virtuální metodu se jménem what, která vrací textový popis výjimky.

```
#include <iostream>
#include <exception>
using namespace std;
class MojeVyjimka : public exception {
    virtual const char* what() const throw() {
        return "Nastala moje výjimka!";
    }
};
int main () {
    try {
        throw MojeVyjimka();
    }
    catch (exception& e) { cout << e.what() << endl; }
    return 0;
}
```

Výstup:
Nastala moje výjimka!

Datové typy výjimek

- Základní datové typy:
 - `int = throw 1; catch (int i) {}`
 - `char = throw 'a'; catch (char c) {}`
 - `char * = throw "vyjimka"; catch (char * vyj) {}`
- Struktura, třída:
 - Definováno plně uživatelem.
 - `struct Vyjimka {} ; Vyjimka vyj;`
 - `throw vyj; catch (Vyjimka &vyj) {}`
- Třídy zděděné z `exception (logic_error, runtime_error)`:
 - `class Vyjimka: public exception {}`
 - `catch (exception & ex) {}`

Datové typy výjimek - příklad

```
struct Vyjimka {
    std::string duvod;
    int hodnota;
};

class cVyjimka : public exception {
public:
    cVyjimka(char* duvod):exception(duvod) {}
};

void fn(int volba) {
    if (volba == 0) throw 0;
    if (volba == 1) throw 'a';
    if (volba == 2) {
        Vyjimka vyj;
        vyj.duvod = "TEST";
        vyj.hodnota= 1;
        throw vyj;
    }
    if (volba == 3) throw
        cVyjimka("Testtrida");
}
```

```
int main() {
    cout << "Pred testovanim vyjimek";
    try {
        fn(3);
        cout << "Nebude nikdy zobrazeno";
    }
    catch (int v) {
        cout << "Int vyjimka: ";
    }
    catch (char v) {
        cout << "Char vyjimka: ";
    }
    catch (Vyjimka& vyj) {
        cout << "Struct vyjimka: " <<
            vyj.duvod;
    }
    catch (cVyjimka& vyj) {
        cout << "Trida vyjimka: " << vyj.what();
    }
    cout << "Blok po testovani vyjimek";
    return 0;
}
```

Výjimky

- Chyba nemůže být ignorována.
 - Ignorování chyby ukončí proces.
- Funkce, která hlásí chybu, hodí výjimku.
- Funkce, která umí chybu zpracovat, ji chytá.
- Funkce se dají skládat.

```
int main() {  
    try {  
        while (true) {  
            new int[100000000u1];  
        }  
    } catch (const std::bad_alloc& e) {  
        std::cout << "Allocation failed: " << e.what() << '\n';  
    }  
}
```

Výjimky – použití

- Při hození výjimky dojde k přerušení normálního běhu programu.
- Příkazy následující po hození výjimky nejsou provedeny.
 - Destruktory objektů na zásobníku ale zavolány jsou.

```
void throws() {  
    std::vector<int> vec;  
    while (true) {  
        std::string str;  
        throw 1;  
        baz(); // Nezavolá se  
    } // str je destruováno  
} // vec je destruováno  
  
int main() {  
    throws();  
    doesnt(); // Nezavolá se  
}
```

```
void throws() {  
    std::vector<int> vec;  
    while (true) {  
        std::string str;  
        throw 1;  
        baz(); // Nezavolá se  
    } // str je destruováno  
} // vec je destruováno  
  
int main() {  
    try {  
        throws();  
        doesnt(); // Nezavolá se  
    } catch (int) {  
        std::cout << "Vyjimka chycena.\n";  
    }  
    doesnt(); // Zavolá se  
}
```

Výjimky – použití

- Při hození výjimky dojde k přerušení normálního běhu programu.
- Příkazy následující po hození výjimky nejsou provedeny.
 - Destruktory objektů na zásobníku ale zavolány jsou.

<pre>void throws() { std::vector<int> vec; while (true) { std::string str; throw 1; baz(); // Nezavolá se } // str je destruováno } // vec je destruováno int main() { throws(); doesnt(); // Nezavolá se }</pre>	<pre>void throws() { std::vector<int> vec; while (true) { std::string str; throw 1; baz(); // Nezavolá se } // str je destruováno } // vec je destruováno int main() { try { throws(); doesnt(); // Nezavolá se } catch (int) { std::cout << "Vyjimka chycena.\n"; } doesnt(); // Zavolá se }</pre>
--	--

Výjimky – „házení“

- Výjimku hodíme pomocí klíčového slova `throw`.
- Hodit se dá libovolný typ/objekt.
- Obvykle se ale hází nějaký potomek `std::exception`.
- Hozená výjimka přeruší normální běh programu a dojde k tzv. „stack unwinding“. Je to proces, při kterém se postupně ruší objekty na zásobníku a hledá se odpovídající `catch` blok.
- Pokud není nalezen vhodný `catch` blok, dojde k ukončení programu.

Nedoporučujeme	Doporučujeme
<pre>throw 1; throw "chyba"; std::vector<int> vec; throw &vec; throw std::vector<int>{};</pre>	<pre>throw std::runtime_error("oops\n"); throw std::system_error(std::error_code{}, "argh\n"); throw std::future_error(std::future_errc::broken_promise);</pre>

Výjimky – chytání

- Výjimka se chytá pomocí klíčového slova **catch()**.
- Chytat se dá libovolný typ.
- Chytání je polymorfní – pokud chytáme třídu, chytáme i její potomky.
- Kód v catch bloku se provede pouze, pokud je dotyčná výjimka zachycena.

Nedoporučujeme	Doporučujeme
<pre>catch (...) {} // Chytne cokoliv catch (int) {} // Chytne throw 1; catch (char) {} // Chytne throw 'a'; catch (int*) {} // Chytne throw <int></pre>	<pre>catch (const std::exception& ex) {} // Chytne exception a potomky catch (const std::bad_alloc& ex) {} // Chytne bad_alloc a potomky catch (const std::logic_error& ex) {} // Chytne logic_error a potomky</pre>

Výjimky – Pravidla použití

- **VŽDY** výjimky „házejte hodnotou“.
- **VŽDY** chytejte referencí (pokud můžete, `const&`)
- Používejte své výjimky, které dědí z nějaké standardní výjimky.
- Nepoužívejte `catch(...)`.

Výjimky – házení, chytání a polymorfismus

- Jak jsme si řekli, chytání výjimek je polymorfní.
- Házení ale není.
- Co to znamená?

```
class B {};  
class D : public B {};  
  
void f(B& throwable) {  
    throw throwable;  
}  
  
int main() {  
    D status;  
    try {  
        f(status);  
    } catch (D& e) {  
        std::cout << "Chytil jsem D!\n";  
    } catch (...) {  
        std::cout << "To je divny...\n";  
    }  
}
```

```
>>> To je divny...
```

```
class B {  
public: virtual void raise() {throw *this;}  
};  
class D : public B {  
public: virtual void raise() {throw *this;}  
};  
void f(B& throwable) {  
    throwable.raise();  
}  
  
int main() {  
    D status;  
    try {  
        f(status);  
    } catch (D& e) {  
        std::cout << "Chytil jsem D!\n";  
    } catch (...) {  
        std::cout << "To je divny...\n";  
    }  
}
```

```
>>> Chytil jsem D!
```

Řešení chyb

- Řešení chyb je těžké vždy
 - Jediné snadné řešení je napodobit pštrosa.
- Vrstvy mezi aplikační logikou a low level kódem musí být uzpůsobeny ošetřování chyb.
 - Pokud správně nepoužívají chybové příznaky, zavlečou spoustu dalších chyb.
 - Pokud správně nepoužívají a nevracejí návratové hodnoty, je to ještě horší.
 - Pokud správně neřeší výjimky, dojde ke ztrátě zdrojů. (resources)

Řešení chyb (2)

- Každý způsob řešení chyb vyžaduje uzpůsobení rozhraní funkcí.
 - Chybové příznaky zavádějí globální stav a neumožňují paralelizaci
 - Návrátové hodnoty vedou k out parameterům
 - Výjimky mění rozhraní funkcí méně nápadným způsobem

Terminologie (Abrahams)

Existují 3 různé úrovně záruky bezpečnosti.

- **Základní (Basic Guarantee)**
 - Pokud dojde k výjimce, nedojde ke ztrátě zdrojů nebo k porušení invariant.
- **Silná (Strong Guarantee)**
 - Pokud dojde k výjimce, nedojde ke změně interního stavu.
- **Neházející (No-throw Guarantee)**
 - K výjimce nemůže dojít.

Žádná vs Základní záruka

```
int* read_numbers(int n, std::istream&
in) {
    int* numbers = new int[n];
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        in >> numbers[i];
        sum += numbers[i];
    }

    return numbers;
}
```

```
std::unique_ptr<int[]>
read_numbers(int n, std::istream& in) {
    auto numbers = std::make_unique<int[]>(n);
    int sum = 0;
    for (int i = 0; i < n; ++i) {
        in >> numbers[i];
        sum += numbers[i];
    }

    return numbers;
}
```

Práce s výjimkami – předpoklady

- Každá funkce poskytuje alespoň základní záruku vůči výjimkám.
- Pokud funkce neposkytuje alespoň základní záruku, nelze pokračovat.
 - Takováto funkce může přesunout program do špatného vstupu bez ohledu na naše akce.
- Každá funkce poskytuje POUZE základní záruku vůči výjimkám.
 - Pokud nedeklaruje jinak.

Práce s výjimkami – předpoklady

- Každá funkce může vyhodit výjimku.
 - Pokud nedeklaruje jinak.
- Destruktor, swap a přesunující akce jsou no-throw.
 - Házející swap a přesunující akce jsou ospravedlnitelné, destruktory nejsou.
- Operace nad primitivními typy výjimky nehází
 - Sčítání, odčítání, přiřazování...

Jak psát bezpečný kód?

- Vždy kontrolujte návratové hodnoty.
- Zjistěte, které funkce mohou hodit výjimku, a rozmyslete si, jak na ně reagovat.
- Používejte specifikace výjimek.
- Používejte `try{} catch(){}` bloky.

Jak psát bezpečný kód?

- ~~• Vždy kontrolujte návratové hodnoty.~~
- ~~• Zjistěte, které funkce mohou hodit výjimku, a rozmyslete si, jak na ně reagovat.~~
- ~~• Používejte specifikace výjimek.~~
- ~~• Používejte `try{} catch(){}` bloky.~~
- Vždy zachovejte invarianty.
 - I v případě chyby.
- Rozdělte kód na část, která může selhat a která selhat nemůže.
 - V části která selhat může se připravují změny.
 - V části která selhat nemůže se změny ukládají.

Transakce?

Jak psát bezpečný kód?

V části, která selhat může,
se změny připravují.

```
list& operator=(const list& rhs) {  
    list temp(rhs);  
    temp.swap(*this);  
    return *this;  
}
```

V části, která selhat nemůže,
se změny ukládají.

Všeobecná pravidla

- VŽDY poskytněte základní záruku.
- Silnou záruku poskytněte pouze tam, kde je to „přirozené“.
 - Pouze tam, kde se dá implementovat bez zbytečné pesimizace výkonu.
- Zdokumentujte všechny funkce se silnou zárukou.
- Implementujte neházející destruktory!
- Implementujte neházející

```
class list {  
public:  
    list(list&& rhs) noexcept;  
    list& operator=(list&& rhs) noexcept;  
    void swap(list& rhs) noexcept;  
    ...  
};
```

Základní záruka

- Základní záruku nabízejí všechny RAII třídy.
- Zároveň platí, že pokud všechny prostředky spravujete pomocí RAII, máte základní záruku ve funkci zdarma.
- Pozor, pokud ji funkce neposkytuje, nelze ji doimplementovat!

Silná záruka

- Mnoho funkcí umožňuje implementovat silnou záruku „zdarma“.
 - Zdarma z hlediska běhu programu, ne bez úsilí programátora.
- Zpravidla se jedná o případ, kdy se funkce dotkne pouze jednoho zdroje.
 - Například: `vector::push_back`

Silná záruka (2)

- Neměla by se ale implementovat za každou cenu
 - `vector::insert` se dá implementovat se silnou zárukou, ale pouze za cenu zpomalení běhu programu
- Uživatel si ji téměř vždy může doimplementovat sám.
- Funkce se silnou zárukou připomínají ACID transakce.

Příklad – push_back

```
void push_back(const T& t) {  
    if (size < capacity) {  
        // Pokud konstruktor hodí výjimku, nic se nestane  
        new (&buffer[size]) T(t); // Může hodit výjimku  
        size++;                    // Nemůže hodit výjimku  
        // Výsledek: silná záruka zdarma  
    } else {  
        // Viz dále  
        ...  
    }  
}
```

Příklad – push_back

```
void push_back(const T& t) {  
    if (size < capacity) {  
        // Již jsme viděli  
        ...  
    } else {  
        // Připravíme si větší kapacitu  
        auto temp = std::make_unique<T[]>(capacity * 2);  
  
        // Nakopírujeme staré prvky  
        std::copy(buffer.get(),  
                    buffer.get() + size,  
                    temp.get());  
  
        new (&temp[size]) T(t); // Přidáme nový prvek  
        swap(temp, buffer);    // Přebereme nový buffer  
        size++;                 // Nemůže hodit  
        capacity *= 2;          // Nemůže hodit  
    }  
}
```

Cargillův zásobník

- V roce 1994 Tom Cargill prošel implementaci zásobníku a identifikoval v něm množství chyb.
- Implementace byla převzata z časopisu *C++ Report*, kde byla publikována jako modelová implementace.
- Na závěr vyzval širší komunitu, aby přišla s implementací zásobníku, která je bezpečná vůči výjimkám.
- Projdeme si nyní tuto implementaci společně.
 - Schválně kolik najdete chyb.
 - Kód je starý.
 - Velmi, velmi starý.

Cargillův zásobník – kód

<pre>template <class T> class Stack { unsigned nelems; int top; T* v; public: unsigned count(); void push(T); T pop(); Stack(); ~Stack(); Stack(const Stack&); Stack& operator=(const Stack&); };</pre>	Deklarace
---	-----------

- Základní implementace, kterou známe.
- Pointer v na pole prvků, nelems kapacita pole a top, index aktuálního prvku.
- Funkce umožňující sebrat prvek z vrcholu zásobníku, vložit prvek a zjistit kolik je v zásobníku prvků.
- Sada základních operací typu: kopírující operace, destruktory, konstruktor.

Cargillův zásobník – kód (2)

```
template <class T>
Stack<T>::Stack() {
    top = -1;
    v = new T[nelems = 10];
    if (v == 0)
        throw "out of memory";
}

template <class T>
Stack<T>::~~Stack() {
    delete[] v;
}

template <class T>
unsigned Stack<T>::count() {
    return top + 1;
}
```

Implementace

- V dnešním C++ operátor new při nepodařené alokaci hodí výjimku, takže vlastní házení není potřeba.
- Jinak zde není žádný problém.

Cargillův zásobník – kód (3)

```
template <class T>
void Stack<T>::push(T element) {
    top++;
    if (top == nelems - 1) {
        T* new_buffer = new T[nelems += 10];
        if (new_buffer == 0)
            throw "out of memory";
        for (int i = 0; i < top; i++)
            new_buffer[i] = v[i];
        delete[] v;
        v = new_buffer;
    }
    v[top] = element;
}
```

```
template <class T>
T Stack<T>::pop() {
    if (top < 0)
        throw "pop on empty stack";
    return v[top--];
}
```

Cargillův zásobník – řešení

- Zásobník nelze opravit bez změny rozhraní

```
template <class T>
class Stack {
    unsigned nelems;
    int top;
    T* v;

public:
    unsigned count();
    void push(T);
    T pop();
    ...
};
```

Stará
deklarace

```
template <class T>
class Stack {
    unsigned nelems;
    int top;
    T* v;

public:
    unsigned count();
    void push(T);
    void pop();
    T& top();
    ...
};
```

Nová
deklarace

Cargillův zásobník – kód (4)

```
template <typename T>
Stack& Stack::operator=(const Stack& s) {
    delete[] v;
    v = new T[nelems = s.nelems];
    if (v == 0)
        throw "out of memory";
    if (s.top > -1) {
        for (top = 0; top <= s.top; top++)
            v[top] = s.v[top];
        top--;
    }
    return *this;
}
```


Cargillův zásobník – kód (4)

```
template <typename T>
Stack::Stack(const Stack& s) {
    v = new T[nelems = s.nelems];
    if (v == 0)
        throw "out of memory";
    if (s.top > -1) {
        for (top = 0; top <= s.top; top++)
            v[top] = s.v[top];
        top--;
    }
}
```

Výjimky se základní třídou exception

- Základní třída pro výjimky je exception – nejobecnější <exception>

```
class exception {  
public: exception();  
        exception(const char * const &message); // inicializace  
        virtual const char *what() const; // překrývat lze v odvozených  
};
```
- Od ní se odvozují další.
- Pozor platí jen pro MS knihovny VS.
- Jinak třída exception nemá konstruktor pro inicializaci hodnoty výjimky musí se dědit z přímých potomků exception, nebo dalších následovníků:
 - logic_error -> domain_error, invalid_argument, length_error, out_of_range
 - runtime_error -> range_error, overflow_error, underflow_error
 - bad_alloc, bad_cast, bad_exception, bad_typeid

Výjimky z rodin `logic_error` a `runtime_error`

```
int main () {
    vector<int> myvector(10);
    try {
        myvector.at(20)=100;} //
    catch (out_of_range& oor) {
        cerr << "Out of Range error: " <<
        oor.what() << endl;
    }
    return 0;
}

int main () {
    try {
        vector<int> myvector;
        myvector.resize(
            myvector.max_size()+1);
    }
```

```
    catch (length_error& le) {
        cerr << "Length error: " <<
        le.what() << endl;
    }
    return 0;
}

int main() {
    // runtime_error
    try {
        locale testlocale("Something");
    }
    catch(exception &err) {
        cerr<<"Caught "<<err.what()<<endl;
    }
    return 0;
}
```

Hierarchie v dědění výjimek

- Základní třída je exception – lze zachytávat rodičovský typ a tím do něj spadnou i všechny odvozené výjimky.
- Pořadí vyhodnocení výjimek: odpovídající typ a pořadí.
- Doporučení nejdříve v pořadí uvádět potomky, potom předky.

```
class Vyjimka : public std::invalid_argument {
public:
    Vyjimka(char * ret):invalid_argument(ret) {};
};

int main() {
    try {
        throw Vyjimka("TEST");
    }
    catch (std::invalid_argument& ex) {
        cout << "Vyjimka_invalid_argument : " << ex.what();
    }
    catch (Vyjimka & ex) {
        cout << "Vyjimka_Vyjimka: " << ex.what();
    }
    return 0;
}
```

- Výjimka od vlastní třídy nebude nikdy provedena

Speciality výjimek

- Výjimku lze obsloužit v jedné části catch a znovu poslat k řešení do dalších potenciálních výjimek:
 - v těle funkce je obsloužena a postupuje k obsloužení i pro část výjimek v těle main – throw bez parametrů.
 - Výjimky by neměly být posílány mimo Váš kód.
- Pro zachycení všech běžných výjimek slouží syntaxe : `catch(...) {}`
 - Nelze ovšem přistoupit k typu výjimky.
 - Vhodné pro obalení funkce main – nebude neřízeně ukončena.
 - Obalení destruktoru – když je v destruktoru chyba – neuvolní se např. Paměť.
- Při nezachycení výjimky následuje ukončení programu:
 - Neobsluhujeme vůbec výjimky.
 - Ocitne se program mimo blok s obsluhou.

Příklad: New a bad_alloc

```
struct Pamet {  
    double pole [20000];  
};  
int main() {  
    Pamet *pole[20000];  
    int i = 0;  
    try {  
        for (;i<20000;i++) {  
            pole[i] = new Pamet[5000]; // při práci s velkým množstvím  
                                       // paměti je test na 0 nedostatečný  
        }  
    }  
    catch(bad_alloc & ba) {  
        cout<< "malo pameti";  
        for (int j = 0; j<i; j++) delete [] pole[j];  
    }  
    cout<<"ahoj";  
    return 0;  
}
```

Speciality výjimek

- Lze specifikovat seznam výjimek, které mohou být vyvolány pouze v určité metodě.
- V prototypu se uvede seznam argumentů za throw:
 - `double fn (int a) throw(char *,int);` // není podporováno ve VS
- Upozornění pro uživatele a překladač, jaké výjimky se budou zpracovávat.
- Při použití `throw()` v prototypu deklarujeme, že nebude z této funkce spuštěna žádná výjimka.
- Pokud je v metodě vyvolána jiná než deklarovaná výjimka -> volá se metoda `unexpected()` a ta přeruší program.
- Metoda `set_unexpected(funkce)` – nastavuje, která metoda se má spustit.
- Pokud není výjimka vůbec zachycena -> volá se metoda `terminate()` a ta přeruší běh programu.
- Metoda `set_terminate (funkce)` – která funkce se má spustit.

Nezachycená výjimka

```
struct Vyjimka {
    std::string duvod;
    int hodnota;
};
void Konec() {
    cout<<"Nezachycena vyjimka";
    exit(1);
}
void fn(int volba) {
    if (volba == 0) throw 0;
    if (volba == 1) throw 'a';
    if (volba == 2) {
        Vyjimka vyj;
        vyj.duvod = "TEST";
        vyj.hodnota= 1;
        throw vyj;
    }
};
```

```
int main() {
    set_terminate(Konec); // metoda pro
                          // nezachycene vyjimky
    cout << "Pred testovanim vyjimek";
    try {
        fn(1);
        cout << "Nebude nikdy zobrazeno";
    }
    catch (int v) {
        cout << "Int vyjimka: ";
    }
    catch (Vyjimka& vyj) {
        cout << "Struct vyjimka: " <<
            vyj.duvod;
    }
    cout << "Blok po testovani vyjimek";
    return 0;
}
```


Doporučení k výjimkám

- Mocný nástroj, ale je nutné mít velké zkušenosti.
- Chybu v konstruktoru nelze naznačit pomocí návratové hodnoty:
 - Řešení pomocí validační metody, která vše zkontroluje, než zavolá konstruktor.
 - Použití výjimky – zděděné z `invalid_argument`.
- Možnost vzniku `memory leak`:
 - Kód díky výjimce neuvolní dyn. paměť.
 - Proto ve výjimce umístit i kód pro případně uvolnění dyn dat.

Přetypování

- Datový typ uvádí, jak se má rozumět bitové reprezentaci údaje.
- Definována pravidla pro konverzi mezi datovými typy – typový systém.
- C++ je statický typový systém – za překladač je kontrolováno, zda dané typy odpovídají deklarovaným entitám.
- C++ umožňuje i kontrolu za běhu programu = RTTI (Runtime Type Identification):
 - zjištění typu objektu za běhu programu.
- Implicitní přetypování na úrovni jazyka C – short – int – long.
- Explicitní přetypování (casting) – uvedení typu v závorkách.
- C++ využívá dědičnosti – potomek lze přetypovat na rodiče.
- C++ má metody typového systému:
 - `static_cast`, `dynamic_cast`, `reinterpret_cast`, `const_cast`.

Statické přetypování: static_cast

- Kontrola typu během překladač:
 - `static_cast<novy_typ> (stary_typ)` - konverze na jiný typ
 - `float b = static_cast<float> (33)/2;`
- Přetypovat lze u tříd pouze potomek na rodiče.

Dynamické přetypování: dynamic_cast

- Kontrola typu za chodu programu:
 - * `dynamic_cast<novy_typ*> stary_typ` - konverze na jiný typ
- Pokud není povoleno přetypovat vrátí 0, jinak adresu objektu.
- Pravidlo = `stary_typ` je typu `novy_typ`, nebo přímo, nebo nepřímo odvozen z `novy_typ` (potomek-rodic).

```
class A {
public:
    int a;
    virtual void tisk() {
        cout<<"A:a "<<a;
    }
    A(int aa):a(aa) {}
};
class B:public A {
public:
    void tisk() {cout<<"B:a "<<a;}
    void hlava() {cout<<"trida B";}
    B(int aa):A(aa) {}
};
```

```
int main() {
    A *obj1 = new B(1);
    obj1->tisk();
    //obj1->hlava(); nelze A nema metodu
    B * obj2 = dynamic_cast<B*>(obj1);
    obj2->hlava();
    A * obj3 = new A(2);
    B * obj4 = dynamic_cast<B*>(obj3);
    if (obj4!=0) obj4->hlava();
    return 0;
}
```

RTTI

- `dynamic_cast`, operátor `typeid`, třída `type_info`
- Třída `type_info` je vrácena operátorem `typeid` a obsahuje metodu `name()`.
- `typeid(tridaA) == typeid(*ukazatel)` – zda je ukazatel typu dané třídy.
- `typeid(*ukazatel).name()` / zobrazí název třídy daného objektu.
- Pokusit se používat `dynamic_cast` ne `typeid`.
- Časté používání `typeid` – může znamenat špatně navrženou dědičnost.

Typový systém - pokračování

- `reinterpret_cast<novy_typ> (stary_typ)`
 - Změní datový typ bez ohledu na omezení ... velmi nebezpečné!!!
- `const_cast <novy_typ> (stary_typ)`
 - Pokud je potřeba odstranit modifikátor `const` z datového typu.
 - Následně lze měnit data a volat nekonstantní metody objektu.
 - Pouze v případě, že máme kód, který nemůžeme modifikovat a v něm nejsou poskytnuty `const` metody.

```
const A* konstObjekt = new A;  
//konstObjekt->tisk(); // error: nemáme konstantní metody  
A* ObjektA = const_cast<A*> (konstObjekt);  
ObjektA->tisk(); // už lze použít konstantní metodu
```

Výjimky – Problémy

- Cesta kódem není jasná
 - U žádného řádku si nemůžeme být jisti, kam bude pokračovat exekuce.
- Zpočátku jsou neintuitivní.
 - Popravdě, často i později.
- Dlouho byly používány špatně.
 - Stále často jsou.

```
int* nacti_cisla(int n) {  
    int* pole = new int[n];  
    for (int i = 0; i < n; ++i) {  
        pole[i] = nacti_cislo(); /* Tahle funkce rozhodně nehází výjimky.  
                                Doufám. */  
    }  
    return pole;  
}
```

The End